

You Don't Need an RTOS (Part 4)

by Nathan Jones



Mindstorms Engineering, LLC
Embedded Systems Design & Education

At least, probably not as often as you think you do. Using a preemptive RTOS can help make a system schedulable (i.e. help all of its tasks meet their deadlines) but it comes at a cost: "concurrently" running tasks can interact in unforeseen ways that cause system failures, that dreaded class of errors known as "race conditions". In part, this is because humans have difficulty thinking about things that happen concurrently. Using the venerable superloop can help avoid this problem entirely and, in some cases, it can actually make a system work that wouldn't work at all using an RTOS! Additionally, a "superloop" style scheduler has the benefit of being simple enough for you to write yourself in a few dozen lines of code.

If you're unfamiliar with terms like "deadlines", "schedulability", or "worst-case response time", then I would argue you are almost *definitely* using an RTOS prematurely! If you *are* familiar with those terms, allow me to show you a few ways to improve the superloop that bring its performance nearly on par with an RTOS, with no concern about race conditions.

This is the fourth (and final!) in a series of posts lauding the advantages of the simple "superloop" scheduler. Other articles in this series are or will be linked below as they are published.

- [Part 1: Comparing Superloops and RTOSes](#)
- [Part 2: The "Superduperloop"](#)
- [Part 3: Other Cool Superloops and DIY IPC](#)
- Part 4: More DIY IPC (this article)

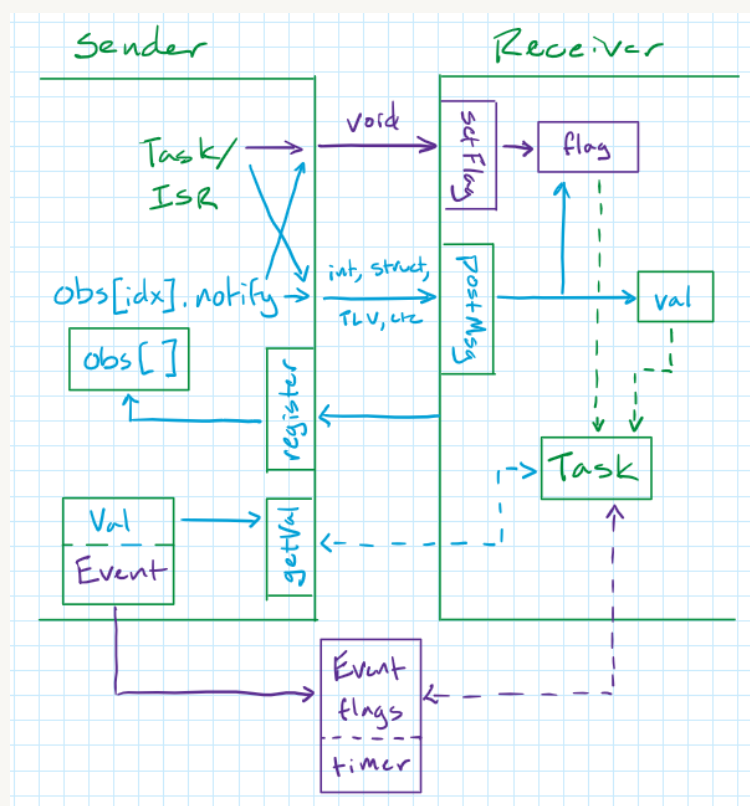
Contents

- [Part 4: More DIY IPC](#)
- [A Quick Recap](#)
- [Mailboxes](#)
 - [Basic Mailbox](#)
 - [Posting Arbitrary Data Types to a Mailbox](#)
 - [Posting Pointers to a Mailbox](#)
- [Queues](#)
 - [Schedulability of a Queue](#)
- [Counting Semaphores](#)
- [The Observer Pattern](#)
- [Marquee](#)

- [Alternative Design: Dispatch Queue](#)
 - [Schedulability of a Dispatch Queue](#)
- [The "Superduperloop", v3](#)
- [Summary of This Article](#)
- [Summary of "You Don't Need an RTOS"](#)
- [References](#)

Part 4: More DIY IPC

In this fourth (and final!) article I'll share with you the last of the inter-process communication (IPC) methods I mentioned in Part 3: mailboxes/queues, counting semaphores, the Observer pattern, and something I'm calling a "marquee". When we're done, we'll have created the scaffolding for tasks to interact in all of the ways depicted below.



Additionally, I'll share with you another alternative design for a non-preemptive scheduler called a **dispatch queue** that is simple to conceptualize and, like the [time-triggered scheduler](#), can help you schedule some of your most difficult task sets.

A Quick Recap

In Part 3 of this series, I shared with you three non-preemptive schedulers that you can potentially use on your next project: ULWOS2, cocoOS, and SST0/QV. Additionally, I demonstrated how we could add thread flags/binary semaphores and event flags to the "Superduperloop". Although it takes a *little* bit of work to add these different forms of IPC to the "Superduperloop" (as opposed to using one of those other three non-preemptive schedulers),

doing so will give you the maximum amount of control over how your scheduler runs, which can be a very useful thing indeed.

Mailboxes

Basic Mailboxes

Mailboxes are a form of point-to-point signaling that would be used when more information than a binary value needs to be communicated; a mailbox (really, just a local variable) is like a queue with room for only one message. Whereas the "setter" functions in the last article took no values (the flag that was being set was determined by the function that was called), the "setter" function for a mailbox will need to know the value(s) that are to be copied into the local variable. The process is essentially the same as for triggering a thread flag or semaphore: another task calls a function which copies the data into the mailbox and sets a thread flag and the task's ready bit, subsequently unblocking the task that is waiting for a message to appear in its mailbox.

```
// TaskA.c
//
enum { MBX_NUM };
const int MBX_MASK = (1<<MBX_NUM);
static volatile int myVal;
void postMsg(int val)
{
    myVal = val; // Assuming storing to myVal is atomic; otherwise,
                // make sure to turn interrupts off before and back
                // on after or use atomic_store(&myVal, val);
    atomic_fetch_or(&thread_flags, MBX_MASK);
    atomic_fetch_or(&g_tasks, A_MASK);
}
void TaskA()
{
    ...
    case STATE_0:
        if( thread_flags & MBX_MASK )
        {
            // Use myVal
        }
    ...
}
```

The "mailbox" here is merely an integer that gets copied to when `postMsg` is called. Like you might expect, writing to the mailbox needs to be atomic, otherwise we'll have a race condition. Here, I'm assuming that writing an integer to memory *is* atomic on whatever processor that's running this scheduler.

Posting Arbitrary Data Types to a Mailbox

We aren't restricted to posting only one type of message to our mailbox, either. We can post *different* types of messages as long as the task has a way of differentiating each message! One such method is [TLV](#) or "tag-length-value". Messages are simply copied, byte-for-byte, into the mailbox (which needs to be large enough to hold the largest message) and the task can decide what to do with the message by looking at the tag value. An example of a task that might benefit from this is a motor controller: different tasks in the system can send different messages to the motor controller that specify speed or direction and the motor controller can set the motor accordingly. One possible implementation of this is shown below.

```
// TaskA.h
//
#define MAX_MSG_SIZE 10
typedef enum
{
    speed_msg,
    direction_msg
} msg_type_t;
// TaskA.c
//
enum { MBX_NUM };
const int MBX_MASK = (1<<MBX_NUM);
typedef struct
{
    msg_type_t tag;
    size_t     length;
    uint8_t    payload[MAX_MSG_SIZE+1];
} mailbox_t;
static volatile mailbox_t mailbox = {0};
void postMsg(msg_type_t tag, size_t length, uint8_t * data)
{
    mailbox_t temp = {0};
    temp.tag = tag;
    temp.length = (length < MAX_MSG_SIZE) ? length : MAX_MSG_SIZE;
    for( size_t idx = 0; idx < temp.length; idx++ ) temp.payload[idx] =
*(data+idx);
    atomic_store(&mailbox, temp);
    atomic_fetch_or(&thread_flags, MBX_MASK);
    atomic_fetch_or(&g_tasks, A_MASK);
}
void TaskA()
{
    ...
    if( thread_flags & MBX_MASK )
    {
        mailbox_t temp = atomic_load(&mailbox);
```

```

        switch( temp.tag )
        {
            case speed_msg:
                ...
            case direction_msg:
                ...
        }
    }
    ...
}

```

It's important to note that atomicity gets weird for structs; we need to do more than just read the struct *members* atomically, we need to read the *entire struct* atomically. That's why I created temp variables in the code above, so that we could use `atomic_store` and `atomic_load`.

Posting Pointers to a Mailbox

If the data to copy is large, you may only want to copy a *pointer* to the data in to the mailbox.

```

static volatile largeStruct_t * mailbox;
void postMsg(largeStruct_t * val)
{
    mailbox = val;
    atomic_fetch_or(&thread_flags, MBX_MASK); // Optional
    atomic_fetch_or(&g_tasks, A_MASK);
}

```

To do so correctly, you'll need to establish if/when a sender is allowed to update the data at that pointer and what the receiver should do (if anything) when its done with the data (primarily to avoid any memory leaks). Those topics, though, will have to wait for [another article!](#)

Copying a pointer also gives you another option for sending arbitrary data to the mailbox: by copying a pointer to a "parent" struct from which different data types "inherit" (by putting the parent struct at the top of their struct definitions). The code below shows a mailbox that accepts a pointer to a generic piece of data which, for the moment, only contains an enumeration.

```

// TaskA.h
//
typedef struct
{
    enum { MYDATA_T } tag
} genericData_t;
// TaskA.c
//
static volatile genericData_t * mailbox;
void postMsg(genericData_t * val)

```

```

{
    mailbox = val;
    atomic_fetch_or(&thread_flags, MBX_MASK);
    atomic_fetch_or(&g_tasks, A_MASK);
}

```

The sender would define a piece of data like this:

```

// Sender.h
//
typedef struct
{
    genericData_t parent;
    int          myInt;
    char         myString[10];
} myData_t;

```

The sender can then create and populate an instance of `myData_t` and post its pointer to the mailbox, since a pointer to `myData_t` *is also a* pointer to `genericData_t` (with some extra stuff after it). The receiver can recover the message based on the value of `tag`, which should be a unique number corresponding to the `myData_t` type, by casting to the correct type.

```

// TaskA.c
//
void TaskA(void)
{
    ...
    if( thread_flags & MBX_MASK )
    {
        mailbox_t temp = atomic_load(&mailbox);
        switch( temp.tag )
        {
            case MYDATA_T:
                myData_t * myData = (myData_t*)&temp;
                ...
            }
        }
    }
    ...
}

```

This is exactly how SST0 and QV handle putting arbitrary events into each tasks' queue. You can read more about this technique at Miro Samek's [website](#).

Queues

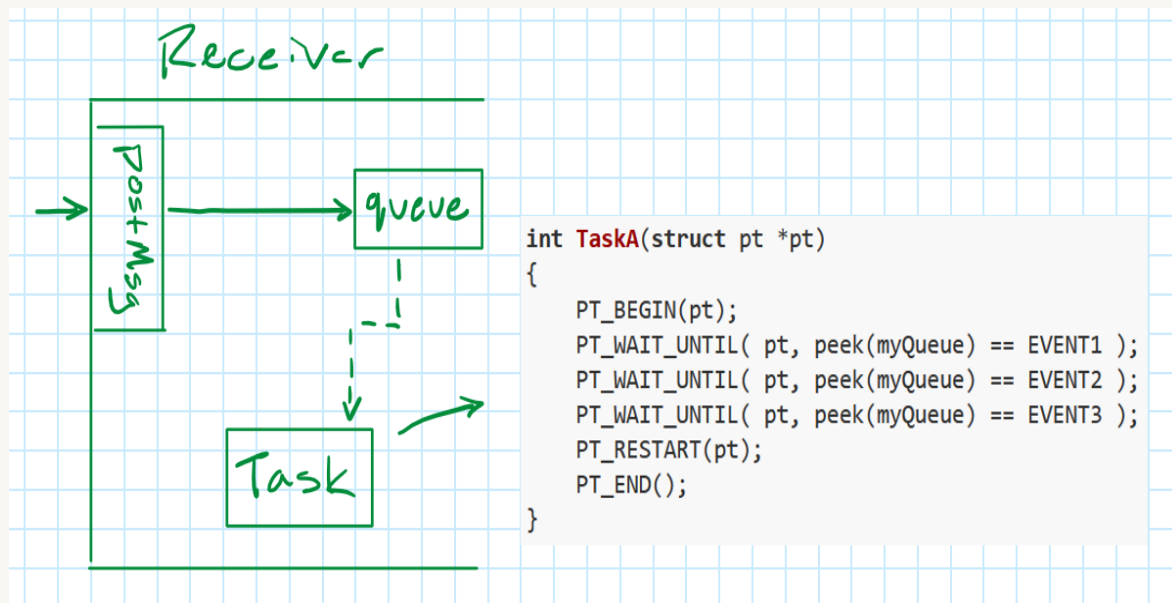
A queue solves the problem of having data arriving more quickly than our task can process it: we can add that value to a queue instead of having to decide whether or not to overwrite our mailbox. Functionally, a queue and a mailbox work the same and everything we've just discussed about mailboxes apply equally well to queues. I'll not try to reinvent the wheel here by creating my own queue implementation, but I'll happily help you find one if you post a request in the comments!

The very thing that makes queues useful, though, is the thing that makes them difficult to analyze! The exact situation we're trying to solve with a queue (or a counting semaphore, see [below](#)) is when events arrive faster than our system can handle them. Queueing them up allows our system to respond more slowly, *eventually* handling all of the events, without needing to respond to each one as it arrives and before the next one shows up. *But this violates our second [critical assumption](#) from the first article!* We would describe that task as having a *short* period (equating to the fastest inter-arrival time of our events) but a *long* deadline (corresponding to the time in the future when all of the events should be handled). Unfortunately, this invalidates the worst-case response time analysis we did in the second article in this series. We'll need to revisit the drawing board, as they say, to determine the schedulability of a system that has tasks with deadlines longer than their periods.

To clarify, just because a task has a queue doesn't *necessarily* mean that our schedulability analysis is invalidated. It's the tasks that use a queue *for the purpose of having a deadline that is longer than their period* that invalidate our previous analysis. The two sample tasks below may help demonstrate this differentiation.

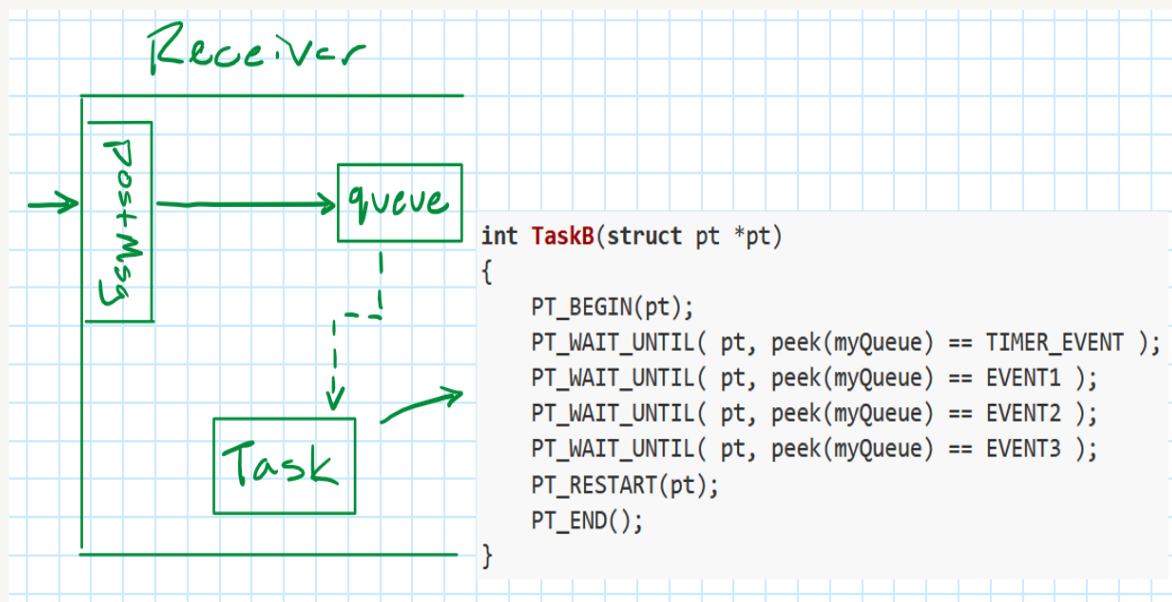
Task A is released by a timer every 500 ms and waits for three separate events before it's finished with its work. Task A uses a queue to store those events when they occur. Task A's deadline is 500 ms because that's when the timer will next have it run. Task A **does not** invalidate our analysis (though if it has to wait a comparatively long time for the events to post to its queue, then our calculation of its WCRT will be optimistically low, since we also assumed in our first article that [tasks don't block each other](#)).

Task	Execution time (ms)	Period (ms)	Deadline (ms)
A	100	500	500



Task B is released by a timer every 500 ms and waits for three separate events before it's finished with its work. All events (including timer events) are enqueued and processed *eventually*; Task B can take up to 3 sec to complete it's work each time the timer releases it. Task B's deadline is 3 sec, which is longer than its period. Task B **does** invalidate our schedulability analysis.

Task	Execution time (ms)	Period (ms)	Deadline (ms)
B	100	500	3000



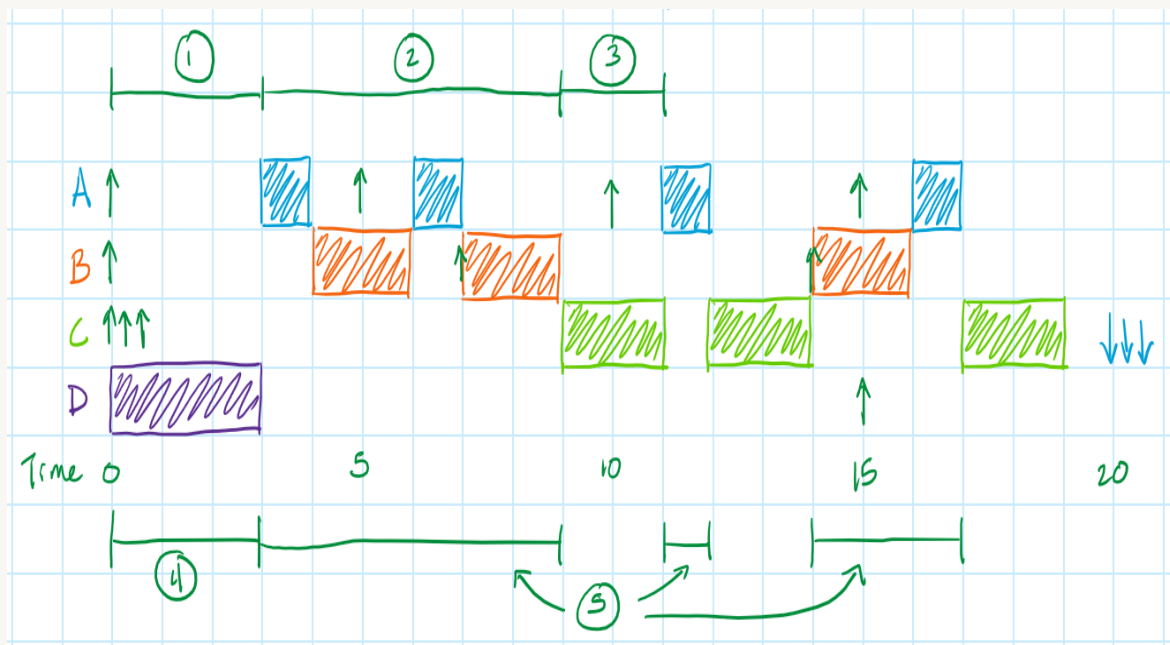
Schedulability of a Task with a Deadline Longer than its Period

So, what's the worst-case schedulability scenario for a task whose deadline is longer than its period? It's when a whole bunch of elements get added to the task's queue at their maximum

rate *juuuust* after the scheduler evaluates its if expression as false. So what's the worst thing that could happen once the queue starts to fill up at its maximum rate?

1. First, a lower priority task runs instead. Only it's not any lower priority task, it the *longest* lower priority task that we have in our system. Once it's done, the scheduler starts back at the top of the loop.
2. Next, *every* higher priority task runs, with the scheduler starting back at the top of the loop after each one.
3. Once no more higher priority tasks are ready to run, the task in question gets to process *the first* event in its queue.
4. Steps 2 and 3 are *repeated* until the queue is empty.

If ISRs are enabled, then the WCRT would also include the total ISR execution time as well. The graphic below depicts how this might play out for an example system. Assume Task C gets released three times at 0 sec.



The WCRT for the *first* event is the normal WCRT for Task C: the total amount of time it would take the system to run the longest, lower-priority task (section 1) plus the execution time of all of the higher-priority tasks and ISRs (section 2) for as many times as they could be released during sections 1 and 2, *plus* the WCET for Task C (section 3). The WCRT for the *q*th event in the queue follows this same pattern: it's the sum of the execution time for the longest, lower-priority task (section 4); the execution time of all of the higher-priority tasks and ISRs (section 5) for as many times as they could be released during sections 4, 5, *and* the first *q*-1 iterations of Task C; and one last iteration of Task C (making a total of *q* iterations of Task C).

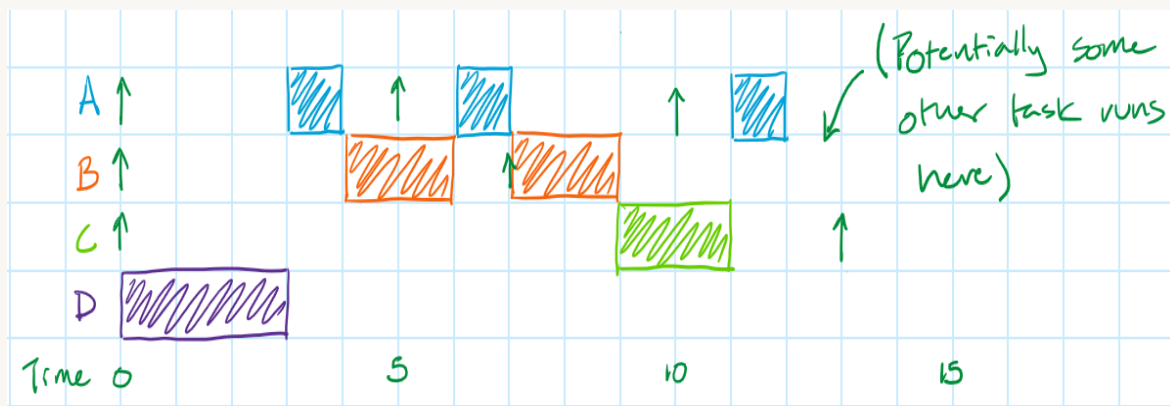
$$WCRT_j, Itemq = \max(WCET_i \in i \> j) + \sum_{i=N_j-1}^1 ([WCRT_j, Itemq - WCET_j P_i] \cdot WCET_i) + \sum_{i=0}^{N-1} ([WCRT_j, Itemq P_i] \cdot WCET_i) + q \cdot WCET_j(1)$$

Furthermore, we can follow the same analysis to arrive at the WCRT for a task with a longer deadline than its period in a preemptive RTOS, also.

$$WCRT_{j,Itemq} = \sum_{i=0}^{q-1} (\lceil WCRT_{j,ItemqPi} \rceil \cdot WCET_i) + q \cdot WCET_j \quad (2)$$

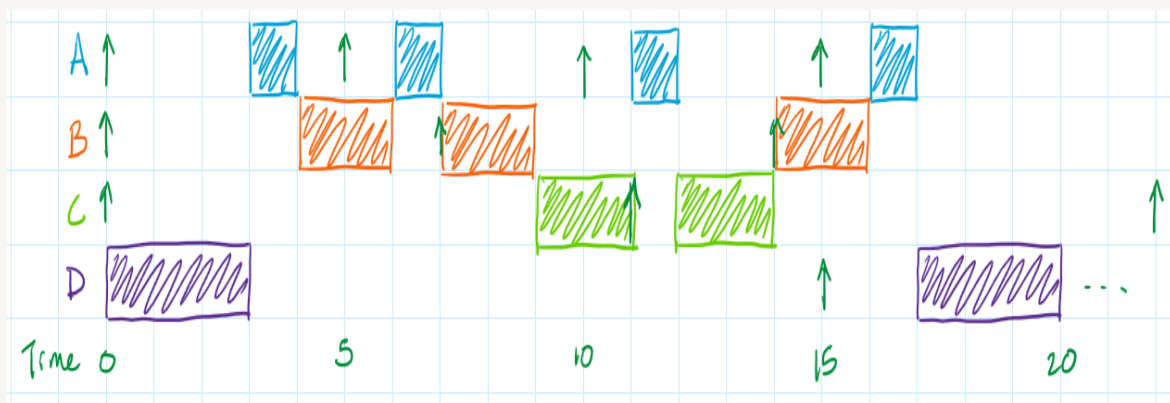
Of course, it doesn't exactly make sense for a task to have its events enqueued instantaneously; that would equate to an inter-arrival time, i.e. a period, of 0 sec! Instead, the events that release the task are (or *should be*) enqueued *no more quickly* than the task's period. How many items a task's queue may need to hold (i.e. the maximum number, N, that we need to consider to see if the task is schedulable) is a function of that period and how quickly the task is able to process each item out of it's queue.

Imagine what would happen with the task set above if Task C's period was actually 13 usec (or anything larger than 12 usec).



In that case, the second event doesn't actually arrive in time for Task C to run it right after the first one and the WCRT for *all* items in the queue (which will only ever have a single element in it) is given by equation 1 with N always equal to 1 (since every event will only ever occupy that first spot in the queue). This, not coincidentally, matches the WCRT we derived in the second article.

If Task C's period was actually something like 11 usec (or anything between 5.67 and 12 usec), then the *second* event *does* arrive to Task C's queue in time to get processed at 12 usec, but the *third* one *doesn't*.



In this case, Task C's queue will only ever have two items in it at any point in time (at most) and the WCRT for each will be given by equation 1, with N equal to either 1 or 2 (corresponding to that event's position in the queue).

In general, equations 1 and 2 only hold for the first q items in a task's queue that each arrive *before* $WCRT_{q+1} - WCET$ (i.e. *before* the *next* time the task would run if there is an item in its queue). If the next event arrives any later, the scheduler will have already skipped over that (since it had no elements in its queue) and it will be like the WCRT analysis starts over. We'll call this maximum value of q , Q .

Additionally, the WCRT of item q in the queue, *isn't actually given by equations 1 or 2!* More accurately, equations 1 and 2 tell us when the *work* for item q is completed.

$$\text{CompletionTime}_{j, \text{Item}q} = \max(WCET_i \in i \text{ and } i > j) + \sum_{i=N_j-1}^{j-1} ([\text{CompletionTime}_{j, \text{Item}q} - WCET_j \cdot P_i] \cdot WCET_i) + \sum_{i=0}^{N-1} ([\text{CompletionTime}_{j, \text{Item}q} \cdot P_i] \cdot WCET_i) + q \cdot WCET_j \quad (3)$$

$$\text{CompletionTime}_{j, \text{Item}q} = \sum_{i=0}^{j-1} ([\text{CompletionTime}_{j, \text{Item}q} \cdot P_i] \cdot WCET_i) + q \cdot WCET_j \quad (4)$$

The *response time* for that item is the *difference* between it's completion time and the time when it was actually added to the queue. The *WCRT* of any task is the *largest response time* of any item that could be added to the queue.

$$\text{ResponseTime}_{j, \text{Item}q} = \text{CompletionTime}_{j, \text{Item}q} - q \cdot \text{Period}_j \quad (5)$$

$$WCRT_j = \max_{q=1 \dots Q} \text{ResponseTime}_{j, \text{Item}q} \quad (6)$$

This WCRT could arise from *any* element in the queue, not necessarily the first or last elements to be added. **So you'll have to check the response times of *all* elements that could be added to the queue at any point in time (i.e. from $q=1$ up to the maximum, Q) to see which is the largest.**

To determine if a task is schedulable, then, you'll iteratively determine the response time (using equation 5) of all the items in a task's queue (starting with $q=1$), with completion times given by either equation 3 or 4, verifying that each item arrives before $WCRT_{q+1} - WCET$. If the q th item *doesn't* arrive before $WCRT_{q+1} - WCET$ (i.e. if when $WCRT_{q+1} - WCET < (q + 1) \cdot \text{Period}_j$), then you've found your maximum queue length, Q (which will be q items). Alternatively, your specific system may dictate a smaller maximum value than this (e.g. "*technically* our queue could have as many as 100 messages before the task is able to clear them out, but it doesn't make sense for there to be more than 10 messages enqueued at any point in time, so *that* will be our maximum value). If the *largest* response time out of *all* of the items in the queue is less than that task's deadline, your task is schedulable!

Since that could be a rather daunting process, I'll offer you one other solution for **preemptive schedulers** based on results from [this paper](#), which puts an upper-bound on the WCRT for any item in a task's queue.

$$WCRT_j \leq \sum_{i \leq j} WCET_i - \sum_{i < j} WCET_j \cdot \text{Period}_j \quad (7)$$

If the WCRT for a task using this equation is less than the task's deadline, then that task is guaranteed to be schedulable. If it isn't, you'll just need to calculate the WCRT for each item in the queue using equation 6, above.

Counting Semaphores

Counting semaphores are used to count the number of times an event occurs. For example, a counting semaphore could be used to count the number of times a button is pressed, with a task running until the count value returns to zero. This would queue up the task however many times the button was pressed, not unlike the way the task might be queued up for however many messages it received in a message queue. Adding support for this just means adding an integer in our task to keep track of the counter value.

```
// TaskA.c
//
static int buttonCount = 0;
void giveSemaphore1(void)
{
    atomic_fetch_add(&buttonCount, 1);
    atomic_fetch_or(&g_tasks, A_MASK);
}
```

The task can then check if buttonCount is non-zero to see if it needs to handle any of those events. (atomic_fetch_sub can be used in the task code to decrement the counting semaphore.)

A counting semaphore, not unlike a queue, though, works on the principle that a task can be released multiple times before it runs, and its okay as long as the task gets to run *eventually*. In effect, we're saying that the deadline of such a task is *longer* than the shortest inter-arrival time of the triggering events, just like with the queue. The schedulability of such a task, then, would follow equation 6, above.

The Observer Pattern

At this point, senders have to know at compile-time which functions they're calling when new data is produced. It would be nice if, instead, this could be set (or even changed) at run-time. The cleanest way to do this is for each sender (or subject) to keep an array of notification functions that each get called when new data is produced. The sender provides register and deregister functions that allow tasks to "sign up" for or "opt out" of notifications.

```
// Subject.h
//
const int MAX_OBSERVERS = 10;
typedef void (*notifyFcn_t)(int val);
// Subject.c
```

```

//
static notifyFcn_t notifyFcns[MAX_OBSERVERS] = {0};
bool register( notifyFcn_t notify )
{
    bool success = false;
    for( int idx = 0; idx < MAX_OBSERVERS; idx++ )
    {
        if( !notify[idx] )
        {
            notify[idx] = notify;
            success = true;
            break;
        }
    }
    return success;
}

void deregister( notifyFcn_t notify )
{
    for( int idx = 0; idx < MAX_OBSERVERS; idx++ )
    {
        if( notify[idx] == notify )
        {
            notify[idx] = NULL;
            break;
        }
    }
}

void SubjectTask(void)
{
    ...
    int newVal = 4;
    for( int idx = 0; idx < MAX_OBSERVERS; idx++ )
    {
        if( notify[idx] ) notify[idx](newVal);
    }
    ...
}

```

Tasks that want to receive notifications from the subject will call the `register` function with a pointer to the function that they want called when new data is ready. That function should copy the data into a local variable (i.e. a mailbox) and then set the thread flags and `g_tasks` to let the task process the data. The notification functions could also screen for specific values, only releasing the receiving task if the new value matches some criteria.

```

// TaskA.c
//
enum { MBX_NUM };

```

```

const int MBX_MASK = (1<<MBX_NUM);
static volatile int myVal;
void receiveVal(int val)
{
    myVal = val;  // Assuming storing to myVal is atomic; otherwise,
                  // make sure to turn interrupts off before and back
                  // on after or use atomic_store(&myVal, val);
    atomic_fetch_or(&thread_flags, MBX_MASK);
    atomic_fetch_or(&g_tasks, A_MASK);
}
void TaskA(void)
{
    ...
    register(receiveVal);  // TODO: Check return value for success/failure
    if( thread_flags & MBX_MASK )
    {
        // If MBX_MASK is set, it's because receiveVal was called
        // by SubjectTask, and myVal has a new value.
    }
    ...
}

```

In effect, this is a simplified version of the [Observer Pattern](#)! The nice part about this is that tasks can now register to receive specific pieces of data when they need them during their execution, instead of needing to wire *every* task at compile-time to receive *every* possible piece of data it could *ever* need throughout the course of its execution. With this, we can build *very* capable systems!

Marquee

"Marquee" is a name I made up for a system in which an **event flag** indicates to the rest of the system that a new **data value** of some kind is available. Tasks that are released when the event occurs can check a global variable or use a "getter" function to obtain the new data value. The task or ISR managing the event also keeps a running counter for the event (to give each event it's own ID number, so-to-speak) so that tasks can check if there's been a new event since last they read the data value.

```

// main.h
//
enum { UART_FLAG_NUM, NUM_EVENT_FLAGS };
const int UART_FLAG_MASK = (1<<UART_FLAG_NUM);
// UART.c
//
typedef struct
{
    char uartRxChar;

```

```

    int  eventCount;
} uartRxStruct_t;
static uartRxStruct_t uartRxStruct = {0};
void UART_ISR(void)
{
    uartRxStruct_t temp = {0};
    temp.uartRxChar = /* Read UART data register */;
    temp.eventCount = uartRxStruct.eventCount + 1;
    atomic_store(&uartRxStruct, temp);
    atomic_fetch_or(&g_event_flags, UART_FLAG_MASK);
    atomic_fetch_or(&g_tasks, tasks_registered_for_events[UART_FLAG_POS];
    // Clear UART interrupt flag
}
uartRxStruct_t getUartResult(void)
{
    return atomic_load(&uartRxStruct);
}
// TaskA.c
static int lastUartCount = -1;
void TaskA(void)
{
    ...
    registerForEvent(A_NUM, UART_FLAG_NUM);
    if( atomic_load(&g_event_flags) & UART_FLAG_MASK )
    {
        uartRxStruct_t uartVal = getUartResult();
        if( lastUartCount != uartVal.eventCount )
        {
            // Do stuff with uartVal.uartRxChar
            lastUartCount = uartVal.eventCount;
        }
    }
    ...
}

```

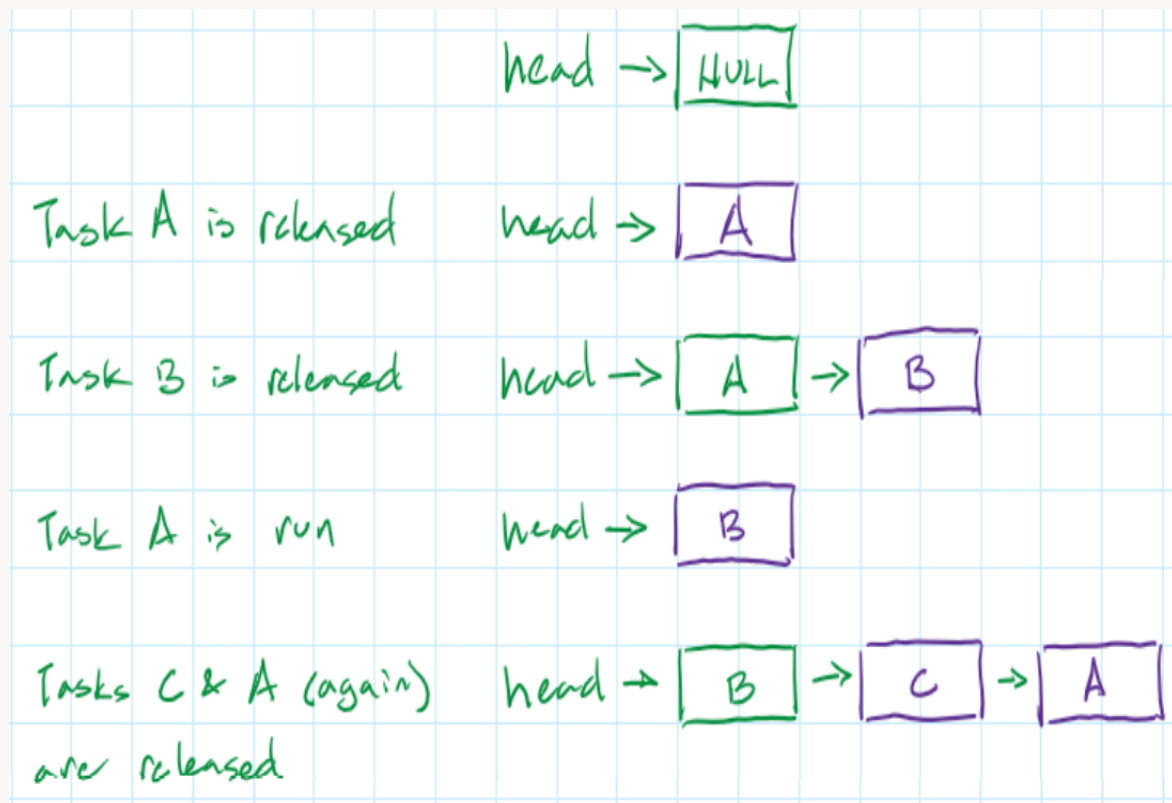
As with mailboxes, the "marquee" isn't restricted to an int or struct; it can also be a TLV message or a pointer (to a larger struct or another generic data type).

Alternative Design: The Dispatch Queue

The "Superduperloop", like many other schedulers, works by looping over a list of tasks in some way, running the first task it finds that's ready to run. Tasks are released by events when their "ready" bit is set (or something similar).

Tasks (in priority order) →				<u>A</u>	<u>B</u>	<u>C</u>
Ready to run? →				X	X	X
① Task A is released				✓	X	X
② Task B is released				✓	✓	X
③ Task A is run				X	✓	X
④ Tasks C & A (again) are released				✓	✓	✓

In a **dispatch queue**, on the other hand, tasks are released by events when *the tasks themselves* (as function pointers, most likely) are added to a system-wide queue. The scheduler does nothing more complicated than dequeue the task at the head of the list and run it; if there are no tasks in the queue, the scheduler runs an idle loop or goes to sleep.



We can still use all of the different forms of IPC that we've discussed over the last two articles by simply replacing `atomic_fetch_or(&g_tasks, A_MASK)` with something like

enqueue(dispatchQueue, TaskA). The main scheduler loop, instead of being composed of an if...else if chain, would look something like this:

```
while(1)
{
    if( queueIsNotEmpty(dispatchQueue) )
    {
        // Current task = dequeue(dispatchQueue)
        // Run task
    }
    else /* Go to sleep */;
}
```

If a normal (i.e. non-prioritized) queue is used to hold each of the tasks, then this amounts to a [first come, first served](#) (FCFS) scheduling algorithm. By enforcing some kind of priority scheme on the queue, though, we can effectively implement different scheduling algorithms (assuming that each task's WCET, period, deadline, and/or it's priority level are stored in the processor's memory). These can include a normal fixed-priority algorithm (like we've been using), [shortest job first](#) (SJF), [earliest deadline first](#) (EDF), or [least laxity](#) (LL). Using a dynamic priority assignment algorithm is nice, because it alleviates any need to determine an optimal priority assignment at compile-time (like we had to do [previously](#)).

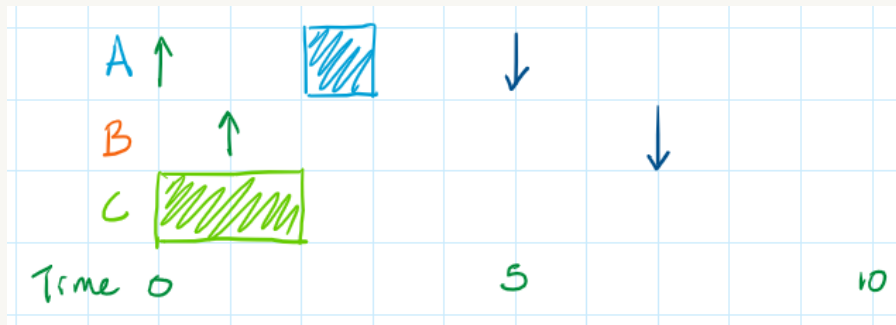
We'll take a closer look at the EDF priority assignment algorithm, specifically, in the next section, since it's been proven to be **optimal** for non-preemptive schedulers. ("Optimal" here doesn't mean that a dispatch queue using EDF can schedule *any* task set, just that if a task set *could* be scheduled by a non-preemptive scheduler, it can *also* be scheduled using EDF.) In fact, [this paper](#) showed that if a task set is schedulable using EDF, you'd have to run your processor either 1.76x or 2x *faster* if you wanted to schedule that exact same task set using a fixed-priority, non-preemptive scheduler like the "Superduperloop"! Despite this, the other algorithms may still warrant consideration if they have better performance in other important categories, such as run-time overhead, how long tasks need to wait (on average or at maximum) before they're allowed to run, etc.

The *downside* with dynamic priority assignment schedulers is that it's unclear which task will suffer if the system is overloaded. With a fixed-priority scheduler, we can be certain that if our tasks actually take longer or arrive more quickly than we'd expected, it's the lower priority tasks which will not get to run. Using a dynamic priority assignment scheduler, which task gets starved in an overload situation is dependent on the specific arrival pattern of the tasks that occurs at run-time.

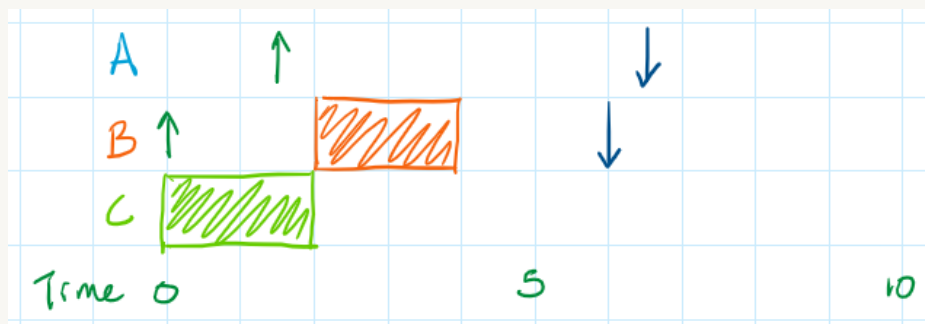
Schedulability of a Dispatch Queue using EDF Priority Assignments

To determine if a set of tasks is schedulable using a dispatch queue with EDF, we'll use a different technique than what we've been using. What we've *been* using is called a "worst-case

response time analysis"; we've mathematically determined the longest time each task could reasonably take to finish its execution (it's WCRT) and said, "As long as *each* task's WCRT is less than or equal to it's deadline, the system must be schedulable." Although it's possible to do a worst-case response time analysis for a dispatch queue with EDF, that analysis gets *mega-weird*, since the "priorities" of each task changes with their release times. Consider two tasks (A and B) with deadlines of 5 and 6 usec. Let's say our system is running a third task when Task A and Task B both arrive (at 0 and 1 usec).



Because Task A has the earlier deadline, when the system finishes the current task, it will be the one to execute first. However, what if the arrival times of Task A and Task B were shifted slightly?



Now *Task B* has the earlier deadline, so *it* will be the one to execute first when the system finishes its current task. This adds a level of complexity to the worst-case response time analysis that makes it quite difficult to get right (at least, for me!).

So instead of a worst-case response time analysis, we're going to use something called a "feasibility" test: if our set of tasks passes the test, then it's schedulable. The feasibility test won't give us any more information than that (so we won't really have any idea about when tasks can be expected to complete before their deadlines, like we might with a worst-case response time analysis), but that is still sufficient for our purposes. (When we used a task set's utilization to determine if it could be scheduled with a preemptive RTOS in the [first article](#), that was an example of a feasibility test, albeit only a sufficient one, not a necessary one.) Further, **we're going to restrict our analysis to task sets in which every task's deadline is equal to its period (or sets in which we make it so by simply taking the lesser of the two values, if they are unequal)**. Note that this means that we're *not* going to analyze any task sets scheduled with EDF that use queues or counting semaphores to have a deadline longer than their period. It's completely possible to analyze those situations, they're just even *harder* to analyze (see the paper below for more information).

Our feasibility test comes from [this paper](#), in which the author's present the following theorem (**WARNING**: GROSS MATH AHEAD):

Theorem 12 - ([KN80], [JSM91]) A non-concrete periodic or sporadic task set (with $D_i = T_i$, $\forall i \in [1, n]$) is feasible, using EDF, if and only if:

$$1) U \leq 1,$$

$$2) \forall t \in S, t \geq h(t) + \max_{D_i > t} \{C_i - 1\},$$

with the convention that $\max_{D_i > t} \{C_i - 1\} = 0$ if $\nexists i: D_i > t$, and

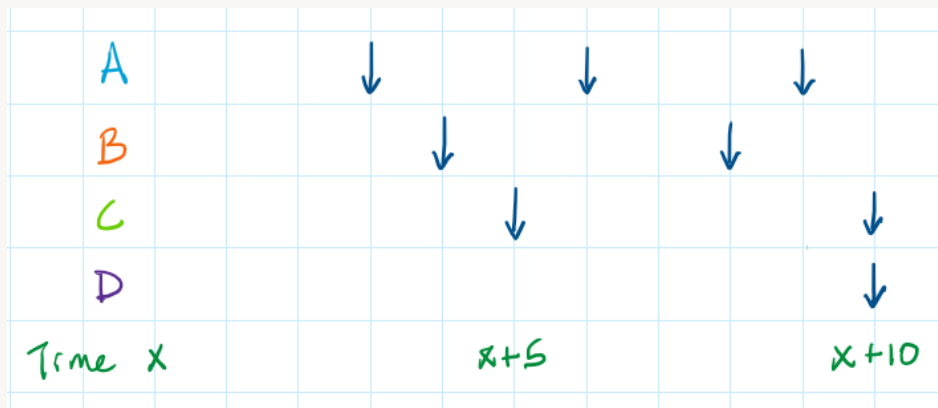
$$S = \left(\bigcup_{i=1}^n \{(k+1)T_i, k \in \mathbb{N}\} \right) \cap [0, \max\{D_i\}],$$

The first condition above says that the total utilization of the processor must be less than or equal to 1 (it's not possible to ask the processor to do 10 usec of work in only 8 usec!). The second condition is the more interesting one. It assumes, firstly, that *all tasks* become ready to run at some time x , which is the worst possible thing to happen in terms of utilization; the processor would have the *most* work to do at any one point in time if that were to happen. Further, it says that if the processor is able to complete all of that work by each tasks' deadlines, then we can say that the system is schedulable (no future arrival pattern could be any worse than this).

To evaluate if that second condition is true for a set of tasks, we'll first construct a set of times (S), starting after x , that correspond to *each* deadline for *each* task up to, and including, the largest deadline. Consider the set of tasks below.

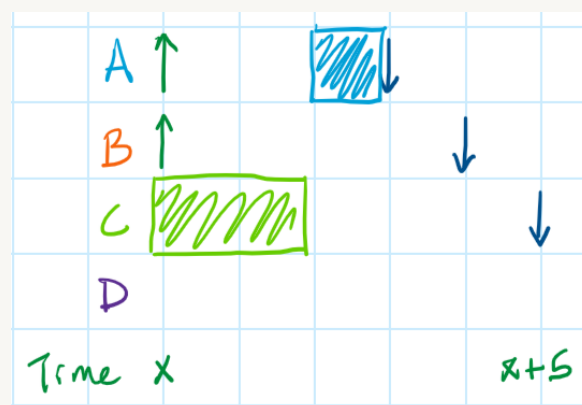
Task	Execution time (us)	Period (us)	Deadline (us)
A	1	3	3
B	1	4	4
C	1	5	5
D	2.1	10	10

If every task were released at x , Task A would have deadlines at $x+3$, $x+6$, and $x+9$; Task B would have deadlines at $x+4$ and $x+8$; Task C would have deadlines at $x+5$ and $x+10$; and Task D would have a deadline at $x+10$. Thus, the set of times we need to consider for our feasibility test are $x+[3,4,5,6,8,9,10]$.

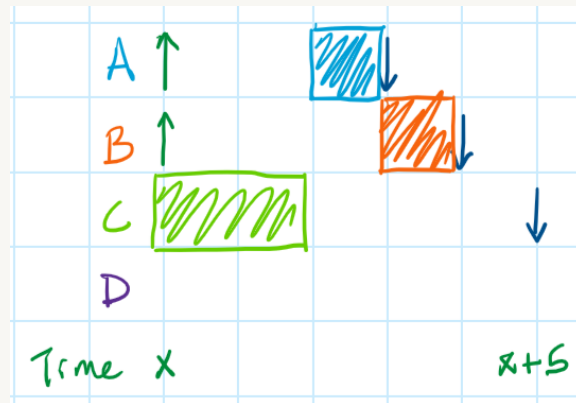


Then, for each of those deadlines, we'll ask the question "Is there enough time for the processor to finish all of its work before that deadline?" The total work that the processor needs to complete ($h(t)$) is the sum of the execution times of all tasks that have a deadline at or before the time in question (t). Note that this would also include the recursive total of all ISRs that could run between x and t . *However*, it's possible that the processor *can't* start working on those tasks at time x , because we're using a non-preemptive scheduler and, in the worst case scenario, it's running another task at x , *right* before every other task gets released. Only (you guessed it!), it's not *any* task that's being run, it's the *longest* possible task that could run whose deadline is *after* the deadline we're currently looking at (at time t); this is the term $\max_{D_i > t} \{C_i - 1\}$. (The term $C_i - 1$ comes from the fact that this paper is doing a discrete time analysis, so tasks are only allowed to be released on integer multiples of the time base of the system. For our purposes, we'll replace $C_i - 1$ with just C_i , since we want to account for the possibility that this task could be released *juuuust* before the other tasks get released [i.e. we want to do a continuous time analysis].) So, in addition to the processor needing to finish all of the work from any task that was released at x and needs to complete by t , it can do that only *after* it finishes the longest task whose deadline is after t . Provided *that's* true for every task in the system, the whole thing must be schedulable.

For example, we'd start by evaluating the first deadline, at $x+3$. At that deadline, only Task A needs to complete and Task C has the longest execution time of any task with a deadline after $x+3$, so the right-hand side of the inequality is $C_A + C_C$ or 3 usec. This just *barely* fits in the 3 usec window, so we're good!



Next, we'd evaluate the deadline at $x+4$. At that deadline, both Task A and Task B need to complete and Task C is still the longest task of any task that has a deadline after $x+4$, to the right-hand side of the inequality is $C_A + C_B + C_C$ or 4 usec. We still barely fit, so we're still good!



The table below shows the results of finishing that analysis for all deadlines up to and including $x+10$.

Deadline	Which tasks have deadlines at or before this time?	Total execution time from all those tasks	Which task (deadline greater than current time) has the longest execution time?	Execution time from that task	Is $t \geq h(t) + \max C_i$?
$x+3$	A	1	C	1	YES
$x+4$	A, B	2	C	1	YES
$x+5$	A, B, C	3	D	2.1	NO
$x+6$	A (twice), B, C	4	D	2.1	NO
$x+8$	A (twice), B (twice), C	5	D	2.1	YES
$x+9$	A (thrice), B (twice), C	6	D	2.1	YES
$x+10$	A (thrice), B (twice), C (twice), D	9	None (no task has a longer deadline)	0	YES

Rats! Our system misses a deadline at 5 usec, owing to the fact that Task D is hogging the processor and doesn't leave it enough time to finish all of its other work. Let's imagine we found a way to reduce the execution time of Task D from 2.1 to 2 usec. The new feasibility test would then look like this:

Deadline	Which tasks have deadlines at or before this time?	Total execution time from all those tasks	Which task (deadline greater than current time) has the longest execution time?	Execution time from that task	Is $t \geq h(t) + \max C_i$?
x+3	A	1	C	1	YES
x+4	A, B	2	C	1	YES
x+5	A, B, C	3	D	2	YES
x+6	A (twice), B, C	4	D	2	YES
x+8	A (twice), B (twice), C	5	D	2	YES
x+9	A (thrice), B (twice), C	6	D	2	YES
x+10	A (thrice), B (twice), C (twice), D	9	None (no task has a longer deadline)	0	YES

Success! Our task set is schedulable using a dispatch queue with EDF.

(Challenge question! Is this task set also schedulable using a preemptive RTOS? What about the "Superduperloop" or the time-triggered scheduler? What are the optimal task priority assignments?)

The "Superduperloop", v3

The final "Superduperloop" (*version 3!*) comes complete with support for all kinds of OS primitives: thread flags/binary semaphores, event flags, mailboxes/queues, Observer, counting semaphores, and marquees. We didn't actually make any changes to the main scheduler loop in this article, so that still looks like it did in the [last article](#).

The tasks, though, were outfitted with additional local flags and variables and with public "setter" or "getter" functions to allow other tasks to set or retrieve those flags/variables. Additionally, "sender" tasks have `register` and `deregister` functions to implement a simple Observer pattern. A sample sender task and a receiver task (unrelated to each other) that demonstrate a few of the important OS primitives are shown below. The "Receiver" task waits for a semaphore (FLAG1), a global event (EVENT_FLAG2), and an integer value from a Subject (myVal). I've chosen to use the [Protothreads](#) library to help keep my FSM syntax succinct (which would also change the signature for the function prototypes in the associated `main.c` file).

```

// Receiver.c
//
//-----Thread flags-----//
enum { FLAG1_NUM, MBX_NUM, NUM_FLAGS_TASK_A };
const int FLAG1_MASK = (1<<FLAG1_NUM);
const int MBX_NUM = (1<<MBX_NUM);
static int thread_flags = 0;
//-----Local variables for mailbox-----//
static int myVal = 0;
//-----"Setter" functions-----//
void setFlag1(void)
{
    atomic_fetch_or(&thread_flags, FLAG1_MASK);
    atomic_fetch_or(&g_tasks, A_MASK);
}
void addMsg(int val)
{
    myVal = val;
    atomic_fetch_or(&thread_flags, MBX_NUM);
    atomic_fetch_or(&g_tasks, A_MASK);
}
//-----Task-----//
int TaskA(struct pt *pt)
{
    float temp;
    PT_BEGIN(pt);
    PT_WAIT_UNTIL( pt, thread_flags & FLAG1_MASK );
    // Do stuff when flag1 has been set
    thread_flags = 0;
    registerForEvent( A_NUM, EVENT_FLAG2_NUM );
    PT_WAIT_UNTIL( pt, event_flags & EVENT_FLAG1_MASK );
    deregisterFromEvent( A_NUM, EVENT_FLAG2_NUM );
    // Do stuff after event flag 2 has been set
    register( addMsg );
    PT_WAIT_UNTIL( pt, thread_flags & MBX_MASK );
    deregister( addMsg );
    // Do stuff with myVal
    PT_RESTART(pt);    // Resume on next task release at "PT_BEGIN"
    PT_END(pt);
}

```

```

// Sender.h
//
const int MAX_OBSERVERS = 10;
typedef void (*notifyFcn_t)(int val);
// Sender.c
//

```

```

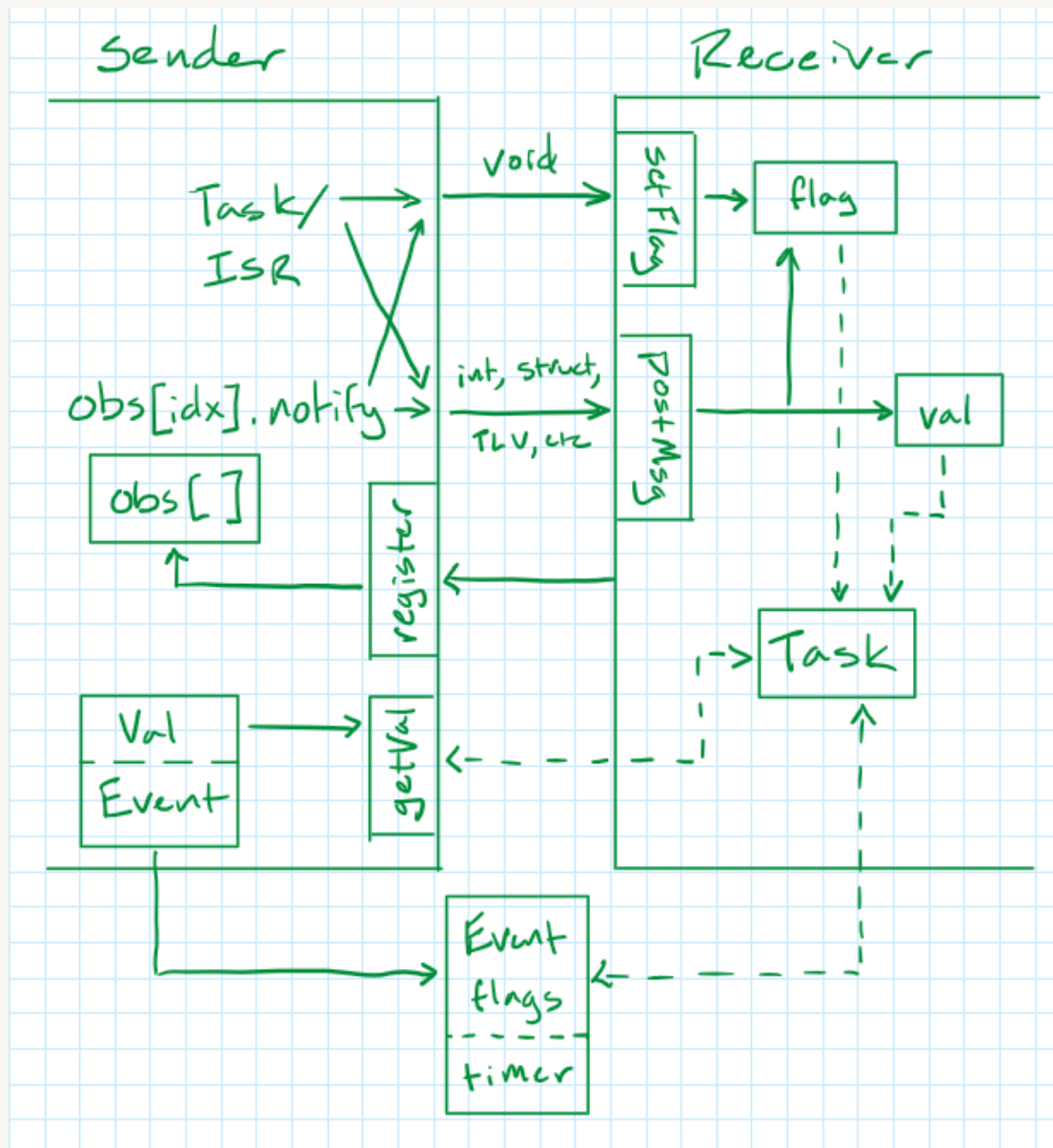
static notifyFcn_t notifyFcns[MAX_OBSERVERS] = {0};
bool register( notifyFcn_t notify )
{
    bool success = false;
    for( int idx = 0; idx < MAX_OBSERVERS; idx++ )
    {
        if( !notify[idx] )
        {
            notify[idx] = notify;
            success = true;
            break;
        }
    }
    return success;
}
void deregister( notifyFcn_t notify )
{
    for( int idx = 0; idx < MAX_OBSERVERS; idx++ )
    {
        if( notify[idx] == notify )
        {
            notify[idx] = NULL;
            break;
        }
    }
}
void adcISR(void)
{
    int newVal = /* Read ADC result register */;
    for( int idx = 0; idx < MAX_OBSERVERS; idx++ )
    {
        if( notify[idx] ) notify[idx](newVal);
    }
    // Clear ADC ISR flag
}

```

Not a bad little OS for only a few dozen lines of code, if I do say so myself!

Summary of This Article

Without too much additional code, we were able to add mailboxes/queues, counting semaphores, the Observer pattern, and marquees to the "Superduperloop". Tasks are able to send both binary and arbitrary data directly to tasks or broadcast them to all tasks, as diagrammed below.



I've updated the tables below that compare the different superloops to reflect these changes.

Advantages and Disadvantages

	Advantages	Disadvantages
ULWOS2	• Simple	• Employs something akin to a dynamic task priority scheme for tasks assigned the same priority level, complicating schedulability analysis • No support for sleeping if idle • Uses gotos to implement coroutine behavior (instead of switch...case), which may be undesirable for some
cocoOS		

	• The most features of any OS here	• Supports sleeping, but with possible error • All messages inherit from a base class (use of inheritance [especially in C!] may be difficult for some)
SST0/QV	• Highly reputable author (Miro Samek) • Likely to be the most stable/bug-free	• Uses message-passing paradigm only (may feel odd to some) • All events inherit from a base class (use of inheritance [especially in C!] may be difficult for some) • SST0 is limited (as written) to only 8 tasks • Not (easily) possible for tasks to yield in the middle of processing an event • SST0 does not currently recycle the memory used to create events (possible memory leak)
"Superduperloop"	• Simple • Supports all IPC • Tasks can wait on any valid C/C++ expression	• Scheduler is so simple b/c much of the IPC code is pushed to the tasks; lots of "boilerplate" code may need to be written

Features

	ULWOS2	cocoOS	SST0/QV	"Superduperloop"
Thread flags	N	N	N	Y
Binary Semaphores	N	Y (but not thread-safe)	N	Y
Event flags	Y	Y	N (but does include timer notifications)	Y
Mailbox/Message queues	Y (but not thread-safe)	Y (but not thread-safe)	Y	Y (with queue library)

Counting semaphores	N	Y (but not thread-safe)	N	Y
Observer	Y	Y	Y	Y
Marquee	Y	Y	N	Y
Complexity	Medium (~500 LOC)	High (~2000 LOC)	- SSTO: Medium (~400 LOC) - QV: High (~1900 LOC)	Low (~50-100 LOC)

Using queues or counting semaphores to enable your tasks to have a deadline which is *longer* than their period complicates the schedulability analysis, though, whether you're using a non-preemptive scheduler or a preemptive one. To determine if your task is schedulable, you'll use equation [6](#) to calculate the task's WCRT, which is the maximum response time of any of the first q items in the task's queue, stopping only when $WCRT_{q+1} - WCET < (q + 1)Period_i$. The response time is given by equation [5](#) and the completion times are given by either equation [3](#) or [4](#) (depending on whether you're using a non-preemptive or a preemptive scheduler). If the WCRT is less than the task's deadline, then the task is schedulable.

Hopefully now you're *even more* ready to write your next embedded application with one of those schedulers, absent (almost) any concerns about race conditions!

Additionally, I demonstrated another scheduler based on the idea of a **dispatch queue**. A dispatch queue is simple to implement and can adopt a number of different task prioritization algorithms based on whether and how you prioritize tasks that get added to the dispatch queue. Using the EDF priority assignment algorithm, you can schedule task sets that even a fixed-priority scheduler like the "Superduperloop" wouldn't be able to schedule.

Summary of "You Don't Need an RTOS"

This being the last article in my series, let's review the main points I had hoped to share with you about real-time scheduling (in general) and also non-preemptive schedulers (in specific).

- If you want to make sure that your system actually works (i.e. that "bad things" aren't happening as a result of tasks missing their deadlines), you need to use a *proven* method of scheduling those tasks with a *proven* way to ensure schedulability.
- Every means of ensuring schedulability will require that you know your tasks' worst-case execution times (WCET), their periods, and their deadlines, which you can either [measure](#) or determine from system requirements.

- Using a preemptive RTOS is *one* way to schedule your tasks, but reasoning about concurrent code and making sure it works correctly is *hard*. It requires considering the possible negative effects (i.e. race conditions) of *any* higher-priority task running between *any* two non-atomic operations in a lower-priority task (and possibly also needing to take into account the reordering of instructions by your compiler or processor!).
- Race conditions can be *eliminated* if a system is non-preemptive. Race conditions can be *minimized* in a system that disallows task preemption, but *does* allow preemption from ISRs.
- Three types of non-preemptive schedulers that you can use to schedule your tasks are:
 - Fixed-priority schedulers, i.e.
 - ["Superduperloop"](#)
 - [ULWOS2](#)
 - [cocoOS](#)
 - [SST0 / QV](#)
 - [Time-triggered scheduler](#)
 - [Dispatch queue](#)
- If you're using a fixed-priority scheduler, you'll need to assign a priority to each task. You can assign task priorities using DMS (deadline monotonic scheduling), provided certain conditions are met. The conditions that must be met to use DMS are based on whether you're using a [non-preemptive](#) or a [preemptive](#) scheduler. Additionally, either an [exhaustive search](#) or [Audsley's algorithm](#) will work for either scheduler.
- As long as our [assumptions](#) hold, then a set of tasks is schedulable if the following conditions are true:
 - **Non-preemptive, fixed priority** (e.g. "Superduperloop", ULWOS2, cocoOS, SST0/QV): Every tasks' WCRT is less than or equal to its deadline. WCRT is calculated using equation [5](#) from part 2.
 - For ULWOS2, the WCRT of any task that shares a priority level with other tasks should be calculated as if that task had the lowest priority of any in that group.
 - For the ["time-triggered"](#) scheduler, each task simply needs to meet its deadline over the entire hyperperiod.
 - **Non-preemptive, EDF** (i.e. dispatch queue): Total processor utilization is less than or equal to 1 and the processor can complete all work by each tasks' deadlines, even if every task is released at the same time AND the processor just began executing the longest task with a longer deadline (described [here](#)).
 - **Preemptive, fixed priority** (e.g. RTOS):
 - Utilization is less than the value given by equation [9](#) from part 1, OR
 - Utilization is less than 100% (provided task periods are all [harmonic multiples](#) of the others), OR
 - Every tasks' WCRT is less than or equal to its deadline. WCRT is calculated using equation [3](#) from part 1.
- The schedulability of an ISR depends on whether or not ISRs can be preempted by other ISRs on your processor, but otherwise follows the worst-case response time analysis

above: if the WCRT of an ISR (as calculated using equation [3 or 4](#) from part 2) is less than its deadline, then the ISR is schedulable.

- ISRs are a form of hardware preemption and, thus, introduce the possibility of race conditions into your code. In general, ISRs should be short, usually only setting a flag or posting a value to a mailbox or queue. Additionally, you'll need to ensure that *no* errors could result from an ISR running in between *any two other instructions* in *any* other part of your code.
- If we allow tasks to have deadlines that are *longer* than their periods, then a set of tasks is schedulable if *these* conditions are true:
 - **Non-preemptive, fixed priority** (e.g. "Superduperloop", ULWOS2, cocoOS, SST0/QV): Every task's WCRT is less than or equal to its deadline. A task's WCRT (equation [6](#)) is the maximum response time of any item that could be added to the queue. Each item's response time is given by equation [5](#) and each item's completion time (on which the response time depends) is given by equation [3](#).
 - **Preemptive, fixed priority** (e.g. RTOS): Every task's WCRT is less than or equal to its deadline. WCRT can be calculated (or upper-bounded) in two ways:
 - WCRT is calculated using equation [6](#), which is the maximum response time of any item that could be added to the queue. Each item's response time is given by equation [5](#) and each item's completion time (on which the response time depends) is given by equation [4](#), OR
 - An upper-bound on WCRT is given by equation [7](#).
- If your system isn't schedulable, you can help make it so by:
 - Running some tasks more often than is needed (especially if this makes the task periods harmonic or makes it such that tasks with [increasing deadlines universally have longer execution times](#))
 - [Reducing a task's WCET](#)
 - Run the CPU at a faster clock speed.
 - Move tasks with shorter deadlines to dedicated co-processors.
 - Move work done by a task onto a hardware peripheral.
 - Look for speed optimizations in your code or toolchain such as loop unrolling, compiling at a higher optimization level, etc.
 - Break up a task into [states](#).
- Lastly, a scheduler is made useful with various forms of [inter-process communication](#) (IPC), which allow tasks to communicate and synchronize with each other. The forms of IPC below were added (without too much difficulty, if I may say so myself!) to the "Superduperloop" and they could likely also be added to the other fixed-priority schedulers I mentioned in the third article (ULWOS2, cocoOS, and SST0/QV) without too much difficulty.
 - [Thread flags/Binary semaphores](#)
 - [Event flags](#)
 - [Mailboxes](#)
 - [Queues](#)

- [Counting semaphores](#)
- [Observer pattern](#)
- [Marquee](#)

I hope you've enjoyed this tour of non-preemptive schedulers and, if you've made it this far, thanks so much for your time and attention! May your tasks always meet their deadlines and your code be race-free!

References

- A lot of my notes come from taking Professor Alex Dean's classes at NC State: ECE560 (Embedded Systems Architectures) and ECE561 (Embedded Systems Design/"Optimization"). His course notes reference, among others, the following articles and books:
 - George, L., Rivierre, N., & Spuri, M. (1996). *[Technical Report RR-2966: Preemptive and Non-preemptive Real-time Uniprocessor Scheduling](#)*. INRIA.
 - Davis, R. I., Thekkilakattil, A., Gettings, O., Dobrin, R., Punnekkat, S., & Chen, J.-J. (2017). Exact speedup factors and sub-optimality for non-preemptive scheduling. *Real-Time Systems*, 54(1), 208–246. <https://doi.org/10.1007/s11241-017-9294-3>
- K. Jeffay, D. F. Stanat and C. U. Martel, "[On non-preemptive scheduling of period and sporadic tasks.](#)" [1991] *Proceedings Twelfth Real-Time Systems Symposium*, San Antonio, TX, USA, 1991, pp. 129-139, doi: 10.1109/REAL.1991.160366.